

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение высшего
профессионального образования

**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТОМСКИЙ
ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Инженерная школа «Интеллектуальные энергетические системы»

Направление 09.03.01 Информатика и вычислительная техника

Отделение интеллектуальных систем

Системы контроля версий

Наименование лабораторной работы

Отчет по лабораторной работе № 2

по дисциплине «ГХИ программирование»

Наименование учебной дисциплины

Выполнил студент гр.
6В51ПИШ

Подпись

06.10.2026

Дата

Д.В.Дьяченко

И.О. Фамилия

Проверил
ассистент (ОИС,
ИШИИЭС)

Должность

Подпись

Дата

Е.С. Михеев

И.О. Фамилия

Томск – 2026 г.

Цель работы:

Получить практический опыт работы с распределённой системой контроля версий Git с использованием облачного сервиса GitHub. Освоить клонирование, коммиты, push и pull, работу с ветками, создание Pull Request, проверку кода, управление задачами и разрешение конфликтов слияния.

Введение

Система контроля версий позволяет сохранять историю изменений файлов, сравнивать версии и восстанавливать предыдущие состояния проекта. Git является распределённой системой: каждый клон содержит локальную историю и позволяет выполнять основные операции без постоянного подключения к серверу. GitHub предоставляет хостинг репозитория, Pull Requests, Issues и Projects. Для выполнения работы написан консольный анализатор чисел на Python, вычисляющий количество, сумму и среднее арифметическое. Работа выполнена без напарника: две роли смоделированы одним студентом в двух локальных копиях; вместо независимого ревью проведена самопроверка.

Ход Работы

Задание

1. Подготовка репозитория: Использован репозиторий diachenko668/lab2-diachenko. Добавлены README.md с описанием проекта и командами запуска, а также .gitignore для временных файлов Python. Адрес репозитория: <https://github.com/diachenko668/lab2-diachenko>.

⋮ Open more actions menu

Latest commit

History

🕒 14 Commits

🕒 14 Commits

Folders and files

Name	Name	Last commit message	Last commit date
📁 docs	📁 docs		
📁 tests	📁 tests		
📄 .gitignore	📄 .gitignore		
📄 README.md	📄 README.md		
📄 SELF_REVIEW.md	📄 SELF_REVIEW.md		
📄 main.py	📄 main.py		
📄 report.txt	📄 report.txt		
📄 shared.txt	📄 shared.txt		

View all files

Repository files navigation

📄 README

More items ▾



Лабораторная работа № 2 — системы контроля версий



Студент: Д. В. Дьяченко, группа 6В51ПИИШ. Репозиторий: <https://github.com/diachenko668/lab2-diachenko>

Рис. 1 репозиторий и файлы учебного проекта на GitHub

2. Клонирование: Репозиторий клонирован по HTTPS командой `git clone`.

Созданы локальные копии `author-a` и `author-b` для индивидуального моделирования параллельной работы.

3. Первый коммит и отправка изменений: В `report.txt` добавлено семь предложений с описанием лабораторной работы. Написаны `main.py` и тесты. Выполнены `git status`, `git add`, `git commit` и `git push origin main`. Коммит 5024689 успешно отправлен на GitHub.

```
Bash / Git — сохранённый журнал выполненных команд

$ git clone https://github.com/diachenko668/lab2-diachenko.git /workspace/lab2-deliverables/author-b
Cloning into '/workspace/lab2-deliverables/author-b'...

Код завершения: 0
$ git commit -m 'Добавлены отчёт, анализатор чисел и тесты'
[main 5024689] Добавлены отчёт, анализатор чисел и тесты
6 files changed, 89 insertions(+), 1 deletion(-)
create mode 100644 .gitignore
create mode 100644 main.py
create mode 100644 report.txt
create mode 100644 shared.txt
create mode 100644 tests/test_main.py

Код завершения: 0
$ git push origin main
To https://github.com/diachenko668/lab2-diachenko.git
64672ba..5024689  main -> main

Код завершения: 0
Нажмите Enter для закрытия журнала...
```

Рис. 2 журналы клонирования, первого коммита и push в main

4. Работа с веткой feature/update: Командой `git switch -c feature/update` создана отдельная ветка. В `report.txt` добавлены ещё три пункта о проверке данных, замечаниях самопроверки и конфликте. Создан коммит `1df89a1`; ветка отправлена командой `git push -u origin feature/update`.

```
Bash / Git – сохранённый журнал выполненных команд

$ git switch -c feature/update
Switched to a new branch 'feature/update'

Код завершения: 0
$ git commit -m 'Дополнен отчёт тремя пунктами и тестами граничных случаев'
[feature/update 1df89a1] Дополнен отчёт тремя пунктами и тестами граничных случаев
 2 files changed, 39 insertions(+)

Код завершения: 0
$ git push -u origin feature/update
remote:
remote: Create a pull request for 'feature/update' on GitHub by visiting:
remote:   https://github.com/diachenko668/lab2-diachenko/pull/new/feature/update
remote:
To https://github.com/diachenko668/lab2-diachenko.git
 * [new branch]      feature/update -> feature/update
branch 'feature/update' set up to track 'origin/feature/update'.

Код завершения: 0
Нажмите Enter для закрытия журнала...
```

Рис. 3 создание ветки feature/update, коммит и отправка на GitHub

5. Создание Pull Request: Создан PR № 2 из feature/update в main:

<https://github.com/diachenko668/lab2-diachenko/pull/2>. В описании указаны изменения и результаты тестирования. Независимого ревьюера нет, поэтому запрос ревью соседу не выполнялся; вместо него опубликован комментарий автора с замечаниями самопроверки.

ЛР № 2: дополнение отчёта и исправление анализатора - #2

#2
Merged
diachenko668 merged 2 commits into
main diachenko668/lab2-diachenko:main from
feature/update diachenko668/lab2-diachenko:feature/update Copy head branch name to clipboard
Oct 6, 2026
Conversation Commits (2) Checks Files changed
Merged

ЛР № 2: дополнение отчёта и исправление анализатора #2
diachenko668 merged 2 commits into
main diachenko668/lab2-diachenko:main from
feature/update diachenko668/lab2-diachenko:feature/update Copy head branch name to clipboard

Conversation

@diachenko668

@diachenko668 diachenko668 commented Oct 6, 2026

...
Owner

Изменения

В report.txt добавлены три пункта. Добавлены тесты пограничных входных данных, исправлены пустой список, NaN/Infinity и переполнение суммы. Ошибки CLI выводятся через argparse с кодом 2 без traceback.

Проверка

python3 -m unittest discover -s tests -v: 9 тестов, все пройдены.
До исправления: 8 тестов, 5 проваленных подслучаев и 1 ошибка.

Ревью

Работа выполняется без напарника. Замечания самопроверки описаны в SELF_REVIEW.md.
Независимое одобрение отсутствует; собственный PR не выдаётся за одобренный другим студентом.

diachenko668 added 2 commits October 6, 2026 16:01

Рис. 4 Pull Request № 2 из feature/update в main

6. Исправление замечаний и слияние: Расширенные тесты выявили деление на ноль при пустом списке, а также отсутствие проверки NaN и Infinity. Добавлены проверки конечности входных чисел и суммы, обработка ошибок CLI. Исправления сохранены в коммите 2a67f2f. После исправлений все 9 тестов пройдены; PR № 2 слит в main. Замечания и исправления описаны в SELF_REVIEW.md.

Самопроверка вместо независимого ревью (напарника нет).

Замечания: пустой список вызывает деление на ноль; NaN/Infinity не отклоняются; CLI должен завершаться с кодом 2 и понятным сообщением; требуется проверка переполнения суммы. Регрессионные тесты воспроизвели эти ошибки.

Исправления уже включены в коммит [2a67f2f](#). Проверено: 9 тестов проходят. Это комментарий автора, а не одобрение PR другим участником.



[@diachenko668](#)

[diachenko668](#) merged commit [6230e09](#) into main [Oct 6, 2026](#)

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

Reviewers

No reviews

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

1 participant

[@diachenko668](#)

Footer



© 2026 GitHub, Inc.

Рис. 5 комментарий самопроверки в Pull Request

```
Bash / Git — сохранённый журнал выполненных команд

$ python3 -m unittest discover -s tests -v
test_cli_invalid (test_main.StatsTests.test_cli_invalid) ... ok
test_cli_valid (test_main.StatsTests.test_cli_valid) ... ok
test_empty_input (test_main.StatsTests.test_empty_input) ... ok
test_fractional_numbers (test_main.StatsTests.test_fractional_numbers) ... ok
test_negative_numbers (test_main.StatsTests.test_negative_numbers) ... ok
test_nonfinite_input (test_main.StatsTests.test_nonfinite_input) ... ok
test_overflowing_total (test_main.StatsTests.test_overflowing_total) ... ok
test_positive_numbers (test_main.StatsTests.test_positive_numbers) ... ok
test_single_number (test_main.StatsTests.test_single_number) ... ok

-----
Ran 9 tests in 0.122s

OK

Код завершения: 0
$ python3 main.py 2 4 6
Количество: 3
Сумма: 12
Среднее: 4

Код завершения: 0

Нажмите Enter для закрытия журнала.
```

Рис. 6 успешное выполнение девяти тестов и запуск программы

7. Создание Issue: Создана задача № 1 «Добавить диаграмму в отчёт», назначенная на diachenko668, с меткой enhancement. Добавлены диаграммы docs/git-workflow.png и docs/git-workflow.svg. После выполнения задачи Issue закрыт. Адрес: <https://github.com/diachenko668/lab2-diachenko/issues/1>.

Добавить диаграмму в отчёт #1

New issue

Copy link

New issue

Copy link

Closed

Closed

Добавить диаграмму в отчёт#1

Copy link

Assignees

diachenko668

Labels

enhancementNew feature or requestNew feature or request

Description

@diachenko668

diachenko668

opened on Oct 6, 2026

Issue body actions

Задача

Добавить диаграмму ветвления и разрешения конфликта в материалы лабораторной № 2.

Критерии готовности

- ☒ Диаграмма показывает базовый коммит, feature/update и роли A/B.
- ☒ Отмечены конфликт shared.txt и объединение изменений.
- ☒ Диаграмма доступна в репозитории и включена в отчёт.

Исполнитель: @diachenko668. Работа индивидуальная; роли выполняются в двух копиях.

Reactions are currently unavailable

Activity

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

Metadata

Metadata

Assignees

Рис. 7 Issue № 1 с исполнителем и меткой enhancement

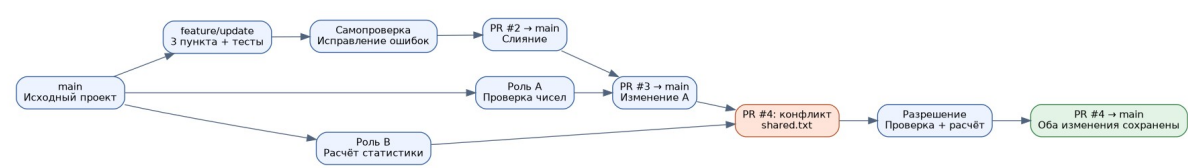


Рис. 8 диаграмма ветвления и разрешения конфликта

8. Работа с Projects: При попытке создать Project GitHub вернул ошибку Resource not accessible by integration (createProjectV2). Доска и перемещение карточки в Done не выполнены из-за отсутствия разрешения у интеграции. Для завершения нужно создать в браузере Project типа Board, настроить колонки To Do, In Progress, Done, добавить Issue № 1 и переместить его в

Done. Инструкция сохранена в docs/PROJECTS_PENDING.md; после выполнения требуется добавить снимок доски в отчёт.

9. Моделирование совместной работы и разрешение конфликта: В копии роли А создана ветка feature/author-a, в копии роли В — feature/author-b. Обе роли изменили одну строку shared.txt: А добавила проверку чисел, В — расчёт статистики. После слияния изменений А с веткой В возник настоящий конфликт; Git сообщил CONFLICT и присвоил файлу статус UU. Маркеры <<<<<<, ===== и >>>>>> удалены. В итоговой строке объединены обе функции: проверка корректности чисел и расчёт статистики. Создан коммит 92712c2 и выполнен push. PR № 3 и № 4 слиты в main после синхронизации ветки В с актуальной основной веткой. Обе роли выполнены одним студентом, второй участник не добавлялся.

```
Bash / Git — сохранённый журнал выполненных команд

$ git merge --no-ff FETCH_HEAD -m 'Интеграция перед разрешением конфликта'
Auto-merging shared.txt
CONFLICT (content): Merge conflict in shared.txt
Automatic merge failed; fix conflicts and then commit the result.

Код завершения: 1
$ git diff -- shared.txt
diff --cc shared.txt
index b84666a,e045dc2..0000000
--- a/shared.txt
+++ b/shared.txt
@@@ -1,3 -1,3 +1,7 @@@
Проект: учебный анализатор чисел.
+++++ HEAD
+Режим обработки: расчёт статистики (роль В).
+++++
+Режим обработки: проверка корректности чисел (роль А).
+++++ 6df654b96641af9819b4365da3a0d1d9a5a15bf5
Участник: Д. В. Дьяченко; роли А и В выполняются индивидуально.

Код завершения: 0

Нажмите Enter для закрытия журнала.
```

Рис. 9 конфликт shared.txt и маркеры конфликтующих изменений

```

Bash / Git — сохранённый журнал выполненных команд

$ git commit -m 'Разрешён конфликт shared.txt: объединены требования ролей А и В'
[feature/author-b 92712c2] Разрешён конфликт shared.txt: объединены требования ролей А и В

Код завершения: 0
$ git push origin feature/author-b
To https://github.com/diachenko668/lab2-diachenko.git
   c4cf80b..92712c2  feature/author-b -> feature/author-b

Код завершения: 0
$ git merge --no-ff origin/feature/author-b -m 'Локальная проверка слияния исправленной ветки роли В'
Merge made by the 'ort' strategy.
 shared.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

Код завершения: 0

Нажмите Enter для закрытия журнала.

```

Рис. 10 фиксация разрешения конфликта и отправка исправленной ветки

10. Подготовка иллюстраций: В отчёт добавлены изображения выполненных этапов. Страницы GitHub загружены с проверкой TLS; стили CDN недоступны, поэтому снимки отображают страницы без оформления сайта. Иллюстрации команд — снимки тёмного окна терминала Bash с сохранёнными фактическими журналами выполнения, приложенными в docs/evidence. Снимок доски Projects отсутствует, так как доска ещё не создана.

11. История коммитов и итоговая проверка: После слияний локальная main обновлена командой `git pull --ff-only origin main`. В основной ветке повторно выполнены 9 тестов без ошибок. На странице истории видны коммиты проекта, feature/update, исправления, изменения обеих ролей, разрешение конфликта и слияния PR. История:

<https://github.com/diachenko668/lab2-diachenko/commits/main/>.

Commits

Branch selector



User selector



Datepicker



Commit history



Commits on Oct 6, 2026

- Добавлена диаграмма ветвления и инструкция для Projects

 diachenko668diachenko668
committed
63b644a [View commit details](#)
 Copy full SHA for 63b644a
 Browse repository at this point

- Merge pull request #4 from diachenko668/feature/author-b

 Show description for 2005378
 diachenko668diachenko668
authored
2005378 [View commit details](#)
 Copy full SHA for 2005378
 Browse repository at this point

- Синхронизация роли В с main после слияния PR A

 diachenko668diachenko668
committed
3ea489d [View commit details](#)
 Copy full SHA for 3ea489d

Рис. 11 страница истории коммитов на GitHub

Контрольные вопросы

Что такое система контроля версий? Какие задачи она решает? Система контроля версий сохраняет историю изменений файлов и позволяет получать предыдущие состояния проекта. Она помогает сравнивать версии, определять авторов изменений, работать параллельно и объединять результаты, а также восстанавливать удалённый или ошибочно изменённый код.

В чём отличие распределённой Git от централизованной SVN? В

распределённой Git каждый клон содержит локальную историю репозитория и позволяет создавать коммиты, ветки и выполнять сравнения без подключения к серверу. В централизованной SVN основная история хранится на сервере; рабочая копия не является полноценным независимым хранилищем истории, а многие операции зависят от доступа к серверу.

Опишите жизненный цикл файла в Git. Новый файл имеет состояние `untracked`. Команда `git add` записывает его снимок в индекс (`staged`), а `git commit` сохраняет подготовленный снимок в истории (`committed`).

Дальнейшее редактирование создаёт состояние `modified`; `git add` и `git commit` повторяют цикл. Если файл изменён после `git add`, в индексе остаётся предыдущий снимок, а рабочая копия содержит дополнительные изменения.

Для чего нужны ветки? Приведите workflow. Ветка — перемещаемая ссылка на коммит, позволяющая независимо развивать изменения. Пример: от `main` создать `feature/update`, внести изменения и коммиты, отправить ветку на GitHub, открыть PR в `main`, проверить код и тесты, исправить замечания и выполнить слияние. Основная ветка при этом остаётся отдельной линией разработки.

Что такое Pull Request и чем он отличается от git merge? Pull Request — запрос на включение изменений в другую ветку с описанием, обсуждением, ревью и проверками на платформе GitHub. `git merge` — команда Git, объединяющая истории локальных веток; сама по себе она не создаёт обсуждения, запроса на ревью или страницы GitHub. Слияние PR может выполняться разными методами: `merge commit`, `squash` или `rebase`.

Как разрешить конфликт слияния? Сначала получить актуальные изменения через `git fetch` и выполнить `merge` нужной ветки. При конфликте изучить `git status` и `git diff`, открыть конфликтующие файлы, выбрать или объединить смысл обеих версий, удалить маркеры конфликта. Затем проверить результат и тесты, выполнить `git add` для разрешённых файлов, `git commit` и `git push`. Если слияние нужно отменить до коммита, использовать `git merge --abort`.

Для чего служат Issues и Projects? Issue описывает задачу или дефект: содержит заголовок, обсуждение, исполнителя, метки и состояние `open/closed`. Projects собирает задачи и PR в представления, например канбан-доску, и позволяет отслеживать план, исполнителей и статусы. Заккрытие

Issue и значение поля Status карточки Project — разные состояния; завершённую задачу необходимо переместить в Done отдельно.

Вывод:

В результате выполнения лабораторной работы освоены основные операции Git: клонирование, фиксация изменений, отправка и получение коммитов, ветвление и слияние. Созданы и слиты Pull Requests, выполнена самопроверка кода, создан и закрыт Issue с диаграммой. В двух локальных копиях воспроизведён и разрешён конфликт общей строки. Написанный анализатор чисел прошёл девять тестов. Работа выполнена индивидуально, без независимого ревью. Для полного выполнения задания остаётся создать GitHub Project, переместить задачу в Done и добавить снимок доски.